

Image Data Preprocessing

Import libraries

```
import tensorflow as tf
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from tensorflow.keras.utils import get_file
```

```
from PIL import Image
```

Load the dataset

```
image_url = "https://upload.wikimedia.org/wikipedia/commons/thumb/a/a3/June\_odd-eyed-cat.jpg/640px-June\_odd-eyed-cat.jpg"
```

```
image_path = get_file("cat.jpg", origin=image_url)
```

```
# Load and display the image
```

```
img = Image.open(image_path).convert('RGB')
```

```
plt.imshow(img)
```

```
plt.title("Original Image")
```

```
plt.axis('off')
```

```
plt.show()
```

Original Image



Resizing the Image

Resize the image to (150, 150)

```
resized_img = img.resize((150, 150))
```

Display the resized image

```
plt.imshow(resized_img)
```

```
plt.title("Resized Image (150x150)")
```

```
plt.axis('off')
```

```
plt.show()
```

Resized Image (150x150)



Image Augmentation

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Create an ImageDataGenerator instance with augmentation
parameters

datagen = ImageDataGenerator(
    rotation_range=40, # Random rotation between -40 to 40 degrees
    width_shift_range=0.2, # Horizontal shift by 20%
    height_shift_range=0.2, # Vertical shift by 20%
    shear_range=0.2, # Shear transformations
    zoom_range=0.2, # Zoom in or out
    horizontal_flip=True, # Random horizontal flip
    fill_mode='nearest' # Fill any empty pixels after transformations
)
```

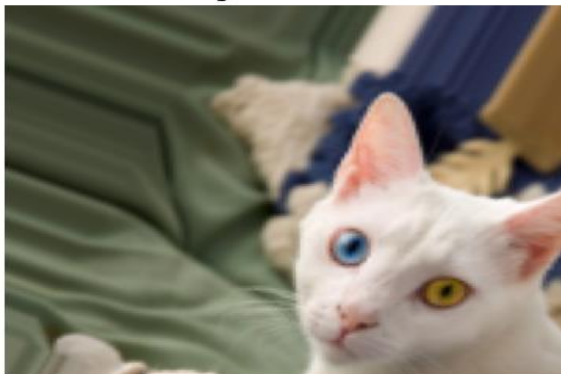
```
# Convert the image array to a batch format (add extra
dimension)
img_batch = np.expand_dims(normalized_img, axis=0)

# Create a 3x3 grid to display multiple augmented images
fig, axes = plt.subplots(3, 3, figsize=(12, 12))

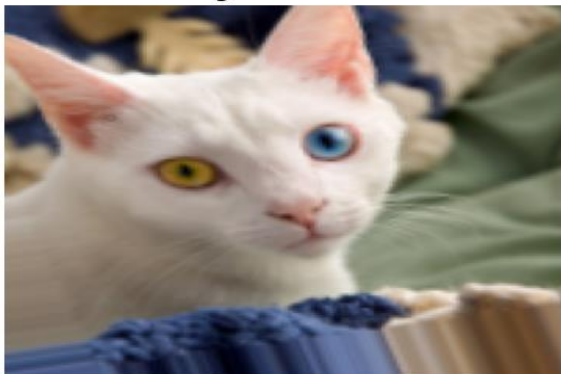
for i, ax in enumerate(axes.flat):
    # Apply a random transformation and visualize the
    result
    augmented_img =
    datagen.random_transform(normalized_img)
    ax.imshow(augmented_img)
    ax.set_title(f'Augmentation {i + 1}')
    ax.axis('off')

plt.tight_layout()
plt.show()
```

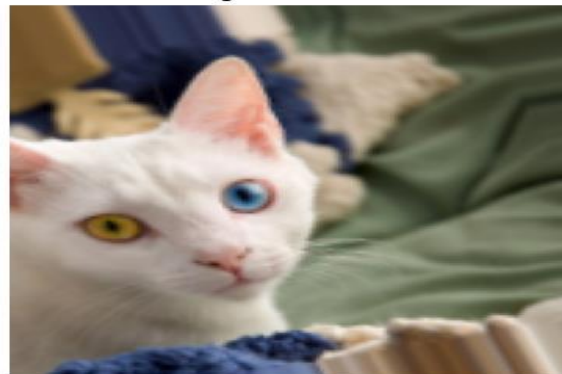
Augmentation 1



Augmentation 2



Augmentation 3



Augmentation 4



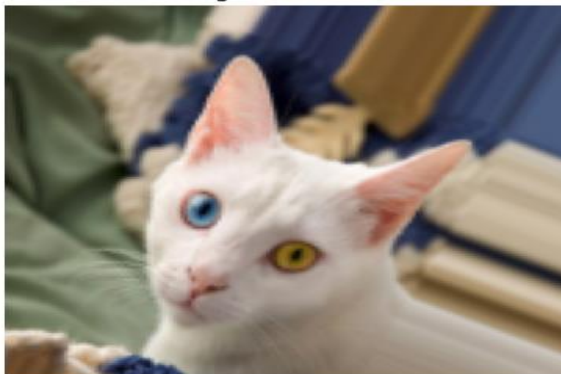
Augmentation 5



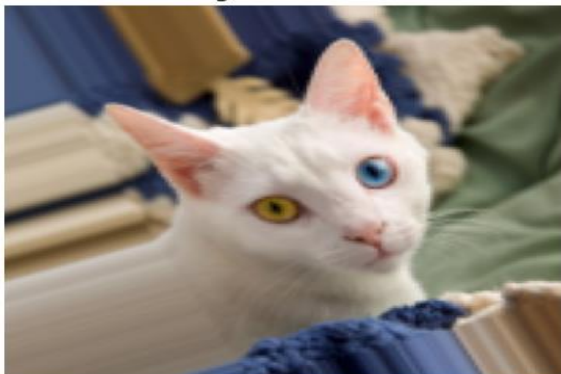
Augmentation 6



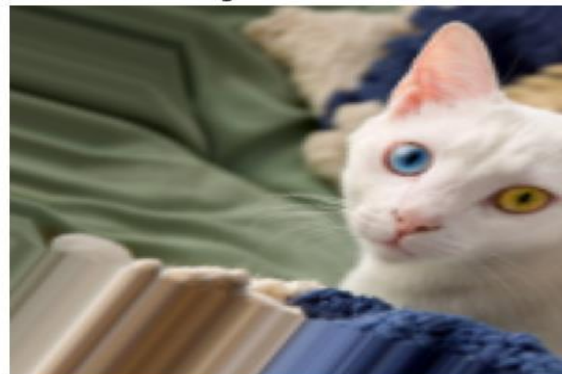
Augmentation 7



Augmentation 8



Augmentation 9



Cropping the Image

```
# Crop the image to the center (150x150  
pixels)
```

```
crop_box = (30, 30, 180, 180) # (left,  
upper, right, lower)
```

```
cropped_img =  
resized_img.crop(crop_box)
```

```
# Display the cropped image
```

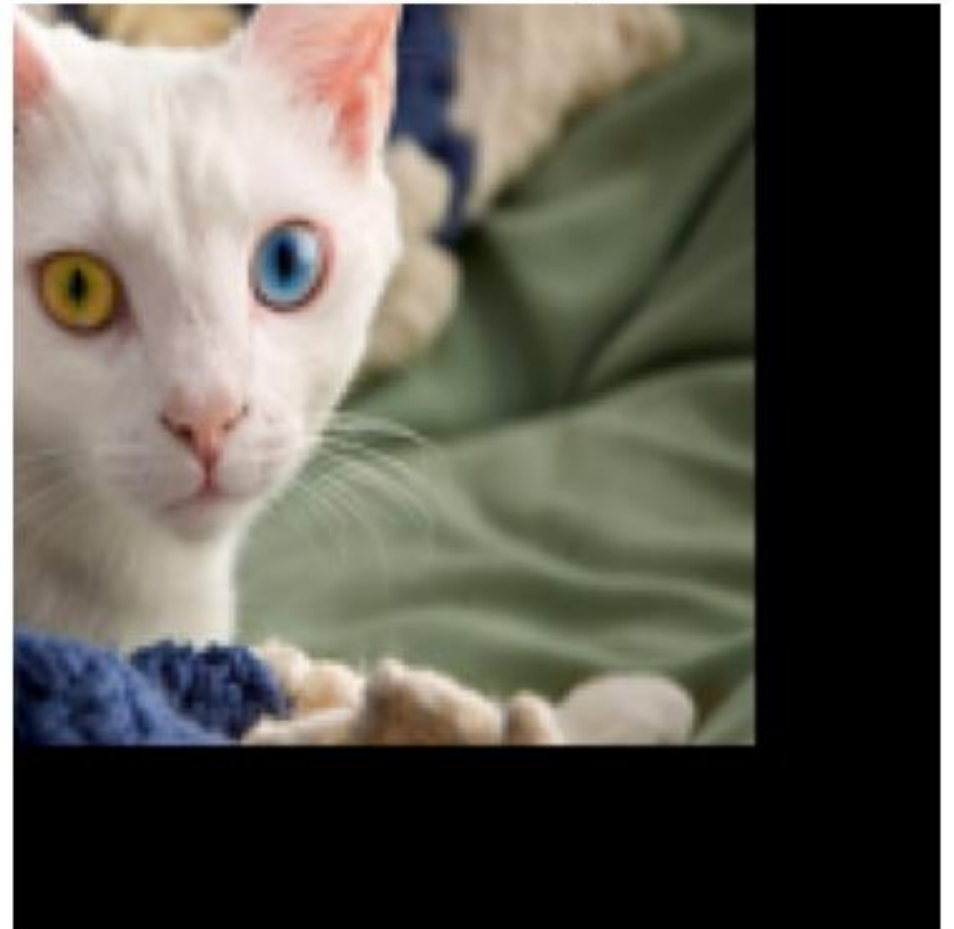
```
plt.imshow(cropped_img)
```

```
plt.title("Cropped Image")
```

```
plt.axis('off')
```

```
plt.show()
```

Cropped Image



Converting to Grayscale

Convert the image to grayscale

```
grayscale_img = resized_img.convert("L")
```

Display the grayscale image

```
plt.imshow(grayscale_img, cmap='gray')
```

```
plt.title("Grayscale Image")
```

```
plt.axis('off')
```

```
plt.show()
```

Grayscale Image



Key Steps for Classification (Binary)

1. Dataset Download and Extraction
2. Data Preparation
3. Model Construction
4. Training the Model
5. Evaluation Metrics

Dataset link: https://storage.googleapis.com/mledu-datasets/cats_and_dogs_filtered.zip

Import libraries

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Dropout
from tensorflow.keras.preprocessing.image import
ImageDataGenerator
from sklearn.model_selection import train_test_split
import os
import zipfile
import requests
```

Step 1: Download and Extract Dataset

```
def download_and_extract_dataset(url, extract_to):  
    filename = url.split("/")[-1]  
    if not os.path.exists(filename):  
        print("Downloading dataset...")  
        response = requests.get(url)  
        with open(filename, "wb") as file:  
            file.write(response.content)  
  
    if not os.path.exists(extract_to):  
        print("Extracting dataset...")  
        with zipfile.ZipFile(filename, 'r') as zip_ref:  
            zip_ref.extractall(extract_to)  
  
dataset_url = "https://storage.googleapis.com/mledu-datasets/cats_and_dogs_filtered.zip"  
extract_path = "cats_and_dogs_filtered"  
download_and_extract_dataset(dataset_url, extract_path)
```

Step 2: Define Dataset Paths

```
base_dir = os.path.join(extract_path, "cats_and_dogs_filtered")  
train_dir = os.path.join(base_dir, "train")  
validation_dir = os.path.join(base_dir, "validation")
```

Step 3: Prepare Data Generators

```
image_size = (150, 150)
```

```
batch_size = 32
```

```
train_datagen = ImageDataGenerator(rescale=1.0 / 255, rotation_range=40, width_shift_range=0.2, height_shift_range=0.2, shear_range=0.2, zoom_range=0.2, horizontal_flip=True, fill_mode='nearest')
```

```
validation_datagen = ImageDataGenerator(rescale=1.0 / 255)
```

```
train_generator = train_datagen.flow_from_directory(
```

```
    train_dir,
```

```
    target_size=image_size,
```

```
    batch_size=batch_size,
```

```
    class_mode='binary')
```

```
validation_generator = validation_datagen.flow_from_directory(
```

```
    validation_dir,
```

```
    target_size=image_size,
```

```
    batch_size=batch_size,
```

```
    class_mode='binary')
```

Step 4: Build the ANN Model

```
model = Sequential([  
    Flatten(input_shape=(150, 150, 3)),  
    Dense(128, activation='relu'),  
    Dropout(0.5),  
    Dense(64, activation='relu'),  
    Dropout(0.5),  
    Dense(1, activation='sigmoid')  
])
```


Step 5: Compile the Model

```
model.compile(optimizer='adam',  
              loss='binary_crossentropy',  
              metrics=['accuracy'])
```

Step 6: Train the Model

```
epochs = 20
history = model.fit(
    train_generator,
    steps_per_epoch=train_generator.samples // batch_size,
    epochs=epochs,
    validation_data=validation_generator,
    validation_steps=validation_generator.samples // batch_size
)
```

Step 7: Evaluate the Model

```
loss, accuracy = model.evaluate(validation_generator)
print(f"Validation Loss: {loss:.4f}, Validation Accuracy:
{accuracy:.4f}")
```

Plot Accuracy Graph

```
plt.figure(figsize=(12, 5))  
plt.subplot(1, 2, 1)  
plt.plot(history.history['accuracy'], label='Training Accuracy')  
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')  
plt.title('Model Accuracy')  
plt.xlabel('Epochs')  
plt.ylabel('Accuracy')  
plt.legend()  
plt.show()
```

Plot Loss graph

```
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.tight_layout()
plt.show()
```

Assignment

- Build a multiclass classification model using an Artificial Neural Network (ANN) to classify the Iris flower dataset into three species: Setosa, Versicolor, and Virginica.
- The dataset contains 150 samples, each with four features: sepal length, sepal width, petal length, and petal width, and the target variable representing the flower species.
- You are required to preprocess the dataset by normalizing the feature values and encoding the labels.
- Split the data into training and testing sets (80% training, 20% testing).
- Design an ANN model with at least one hidden layer, compile it using an appropriate loss function (`sparse_categorical_crossentropy`) and optimizer (`adam`), and train it for a suitable number of epochs.
- Evaluate the model on the test set, report the accuracy, and visualize the training and validation accuracy and loss trends. Additionally, create a confusion matrix to analyze the predictions.

Thank you

